Move from SQL to SOQL

Learning Objectives

After completing this unit, you'll be able to:

- Understand the benefits of the Salesforce objects.
- Identify what is similar and different between SQL and SOQL.
- Build a simple SOQL statement using Workbench.
- Build more complicated relationship queries.
- Build aggregated queries.

Understanding Salesforce Objects

The Salesforce platform provides a powerful database, with many features that make it quick and easy to create applications. Having dealt with SQL Server, you know that data is stored in tables and rows. The database in Salesforce, on the other hand, uses objects to store data. Objects contain all the functionality you expect in a table, with additional enhancements that make them more powerful and versatile. Each object comprises a number of fields, which correspond to columns in a database. Data is stored in records of the object, which correspond to rows in a database. But wait... there's more!

There are two types of objects:

1. **Standard Objects** – These are objects that come baked-in with Salesforce. Some common CRM objects include Accounts, Contacts, Opportunities and Leads.
2. **Custom Objects** – These are new objects you create to store information unique to your application. Custom objects extend the functionality that standard objects provide. For example, if you're building an app to track product inventory, you might create custom objects called Merchandise, Orders or Invoices.

As you probably guessed, objects can have relationship fields that define how records in one object relate to records in another object. These are essentially primary and f keys, but they are much more flexible, making it easier to design and implement data model.

provided by the platform automatically for you. Objects also have built-in support for features such as access management, validation, formulas, and history tracking. All attributes of an object are described with metadata, making it easy to create and modify records either through a visual interface or programmatically.

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Move from SQL to SOQL

As you can see, objects are a lot more than simply containers for storing data. They provide a rich set of functionality that frees you up to focus on building the features unique to your application. For more information on how to create custom objects, fields, relationships and much more, check out the Data Modeling (https://developer.salesforce.com/trailhead/module/data_modeling) module.

Similar but Not the Same

As a .NET developer, chances are pretty high that you're comfortable working with SQL Server. And chances are also high that you're familiar with writing ad hoc queries using SQL. So we thought the best way to introduce you to a similar language designed specifically for Salesforce called SOQL, or Salesforce Object Query Language, was to compare the two.

The first thing to know is that although they're both called query languages, SOQL is used **only** to perform queries with the SELECT statement. SOQL has no equivalent INSERT, UPDATE, and DELETE statements. In the Salesforce world, data manipulation is handled using a set of methods known as Data Manipulation Language (DML). We'll talk more about DML shortly. For now, you just need to know how to query Salesforce data using the SELECT statement that SOQL provides.

One big difference you'll notice right away is that SOQL has a different way of doing `SELECT *` copy. Because SOQL returns Salesforce data, and that data lives in a multi-tenanted environment where everyone is kind of "sharing the database," a wildcard character like `*` copy is asking for trouble. Honestly speaking, it's just too easy to fire up a new SQL query and type in `SELECT * FROM SOME-TABLE` copy, especially when you don't know what the table field names are. This action could severely impact other tenants in your shared environments. To protect the tenant structure and respect field-level security SOQL used `FIELDS()` copy so it only shows the fields that you have permission to access. For example: `SELECT FIELDS(ALL) FROM Account` copy returns all standard and custom fields that you have access to for the Account object.

In SOQL, you specify each field name to return. Beyond that, the SELECT statement that SOQL provides is similar to SQL. You'll find writing SOQL queries pretty easy. But here's what you need to know: Although SQL and SOQL are similar, they're not the same.

Building a Query with Developer Console

Speaking of writing SOQL queries, you might now be thinking, "Well, how do I do that?" One easy way to get started is to use the Salesforce Developer Console (https://help.salesforce.com/s/articleView?id=platform.code_dev_console_tab_query_editor.htm&type=5). We know how much .NET developers love tools, and this powerful tool offers many ways for administrators and developers to access their orgs using Salesforce APIs.As a .NET developer, you're probably familiar with Visual Studio Code. You will be happy to learn that we have Salesforce Extensions for VS Code (https://developer.salesforce.com/tools/extension_vscode) that allow you to do custom development and SOQL building on your local machine. The extension is closely tied to Salesforce DX, which provides a modern source-driven development experience. You can learn more about Salesforce DX from the Get Started with Salesforce DX (https://trailhead.salesforce.com/en/trails/sfdx_get_started) trail.

For now, we'll focus on using Developer Console to build SOQL queries, but if you have some free time, browse around Dev Console and check out all that it offers. We think you'll like it. Sign up for a free Developer Edition (DE) org (https://developer.salesforce.com/signup) and then:

1. From the Setup (⚙) menu, select **Developer Console** to open Developer Console.

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Move from SQL to SOQL

3. Enter the following query:

```
SELECT Id, Name, Type FROM Account
```
copy

4. Click **Execute**.
   The query results include three columns. Scroll through the results.

Notice that the query is getting fields from an object and not a table. Data is persisted inside of objects. Actually, they're called sObjects, as in Salesforce Objects, and they're tightly integrated with the Salesforce platform, which makes them easy to work with.

The bad news here is that datetime fields in Salesforce are just as complicated and hard to work with in SOQL as in SQL. The good news is that Salesforce offers several date functions (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_select_date_functions.htm) that make working with them in SOQL less painful. And while you're poking around the documentation, check out how SOQL handles currency fields (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_querying_currency_fields.htm), because they too are a little different, especially for orgs that deal with multiple currencies.

Once again, the easiest way to see how this works is to use Workbench.

1. From the Setup (⚙) menu, select **Developer Console** to open Developer Console.
2. Click the **Query Editor** tab in the bottom pane.
3. Enter the following query:

```
SELECT AccountId, Email, Id, LastName FROM Contact
```
copy

4. Click **Execute**.

This query returns all contacts in your org. For a development org, this list might be small. But for most real-world orgs, the number of contacts returned could be thousands, which is why you should always consider filtering your SOQL queries with a WHERE clause, especially those used in Apex code.

1. Update the query by adding the WHERE clause:

```
SELECT AccountId, Email, Id, LastName FROM Contact WHERE Email LIKE '%.net%'
```
copy

2. Click **Execute**.

Did you notice what happened? Your constructed query doesn't include the word `contains` copy. Instead, it uses `LIKE` copy. The query also includes the required single quotes (because Email is a text field), as well as leading and trailing percentage signs to indicate that it's a wildcard search.

Note: Using wildcards, especially leading and trailing wildcards such as in the above example, isn't considered good practice. You learn more about how to build efficient queries in a later unit, but for now keep in mind to avoid wildcard searches such as these whenever possible.

1. While you are at it, go ahead and order the results by LastName with the `ORDR BY` copy clause. Your query now looks like this:

```
SELECT AccountId, Email, Id, LastName FROM Contact
```
copy
```
WHERE Email LIKE '%.net%' ORDER BY LastName ASC NULLS FIRST
```
copy

2. Click **Execute** to return the results as an ordered list.

The big thing we want you to notice in the results are the two ID fields: AccountId and Id. These fields contain a unique 18-character string that are assigned by the platform when the Account and Contact records were created. The AccountId field is associated

join tables in SOQL. The short answer is you don't. SOQL has no equivalent JOIN clause. But don't worry, because we think that's a good thing.

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Move from SQL to SOQL

A Different Kind of Join

We're sure you won't be surprised to hear this, but Salesforce does things a little differently when it comes to combining objects, or tables, as you're used to thinking of them. Rather than combine tables with a JOIN clause, you write what is known as relationship queries.

"And what are those?", we can almost hear you saying. Glad you asked.

Salesforce uses a parent-child relationship to combine two objects. Just like in SQL, SOQL uses a foreign key to relate these two objects, but in SOQL, the query syntax is different. At first, using this new syntax probably feels a little weird because you're working with objects instead of rows. But once you get used to the basics, you'll find writing relationship queries much easier than the joins you write in SQL.

SOQL has two basic relationship query types that you need to remember:

- Child-to-parent
- Parent-to-children

Each one works differently. Because you've already seen some fields for the Account and Contact objects, which are most commonly combined, we'll start with them. The important thing to know for now is that Account is the parent, and Contact is the child.

Write a Child-to-Parent Query

Let's say that you wanted to write a query that returns Account and Contact information. Your first option is to write a child-to-parent query. This relationship query uses "dot notation" to access data from the parent, that is, a period separates the relationship name from the name of the field being queried.

To see how this works, we'll walk through writing a relationship query that returns a list of contacts, including the name of the associated account. But this time, instead of using Workbench, we'll use the Query Editor tab in Developer Console (https://help.salesforce.com/s/articleView?id=sf.code_dev_console.htm&type=5).

1. In Developer Console, click the **Query Editor** tab in the bottom pane.

Because we know you're probably still thinking in SQL terms, imagine this scenario. If you had two SQL tables named Account and Contact with a one-to-many relationship between the tables, how would you go about writing a SQL query that returned this same information?

You would use a join, of course. But in this case, you need a right outer join, because you want an equivalent query that returns all contacts, even those not related to an account. And that might look something like the following:

```
SELECT c.FirstName, c.LastName, a.Name
FROM Account a
RIGHT JOIN Contact c ON (c.AccountId = a.Id)
```

So now we want you to go back and look at the equivalent SOQL query:

```
SELECT FirstName, LastName, Account.Name FROM Contact
```

The SOQL query looks much simpler, don't you think?

To be honest, relationship queries can get a little tricky sometimes, especially when figuring out what the relationship name is for custom objects. To learn more about converting SQL queries into SOQL queries, check out this hands-on training video (https://www.youtube.com/watch?v=muYQ6rRLVsY).

One super important thing to be aware of with these types of queries is that you can traverse five levels up with the dot notation. So you can go from the child to the parent to the grandparent to the great grandparent, and so on.

Writing Parent-to-Children Queries

Parent-to-children queries also use the relationship name, but they use it in what is called a nested select query. As with the last query type, it's better to explain this by showing you an example.

This time we'll write a query from the parent Account object and have it include a nested query that also returns information about each associated contact.

1. In Developer Console, click the **Query Editor** tab in the bottom pane.

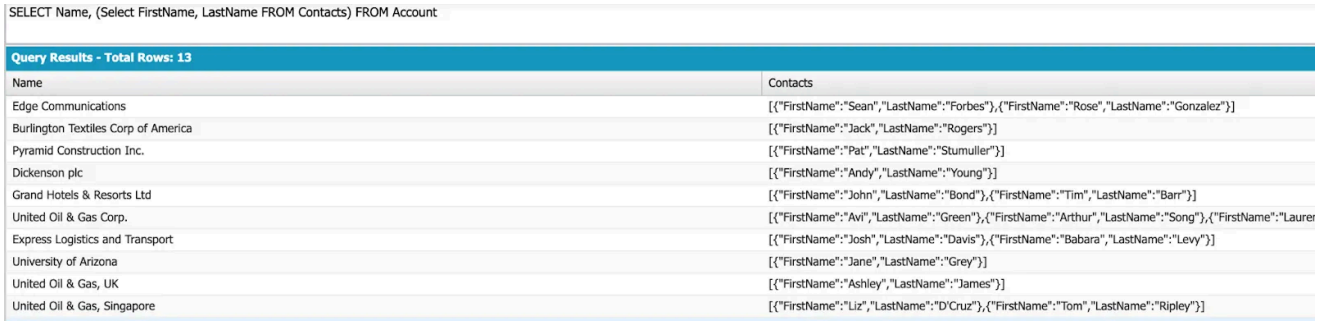
usually trips people up. When working with relationship queries, the parent-to-child relationship name must be a plural name.

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Move from SQL to SOQL

When working with custom objects, the relationship name is not only plural, but it is appended with two underscores and an r. For example, the relationship name for the custom object My_Object__c is My_Objects__r.

3. Click **Execute**.
   The query results include two columns. Notice in the following image how the results under the Contacts column are displayed. Because each account is typically associated with multiple contacts, the first and last names are displayed in JSON format.



SELECT Name, (Select FirstName, LastName FROM Contacts) FROM Account

Query Results - Total Rows: 13

Name	Contacts
Edge Communications	[{"FirstName":"Sean","LastName":"Forbes"},{"FirstName":"Rose","LastName":"Gonzalez"}]
Burlington Textiles Corp of America	[{"FirstName":"Jack","LastName":"Rogers"}]
Pyramid Construction Inc.	[{"FirstName":"Pat","LastName":"Stumuller"}]
Dickenson plc	[{"FirstName":"Andy","LastName":"Young"}]
Grand Hotels & Resorts Ltd	[{"FirstName":"John","LastName":"Bond"},{"FirstName":"Tim","LastName":"Barr"}]
United Oil & Gas Corp.	[{"FirstName":"Avi","LastName":"Green"},{"FirstName":"Arthur","LastName":"Song"},{"FirstName":"Laure
Express Logistics and Transport	[{"FirstName":"Josh","LastName":"Davis"},{"FirstName":"Babara","LastName":"Levy"}]
University of Arizona	[{"FirstName":"Jane","LastName":"Grey"}]
United Oil & Gas, UK	[{"FirstName":"Ashley","LastName":"James"}]
United Oil & Gas, Singapore	[{"FirstName":"Liz","LastName":"D'Cruz"},{"FirstName":"Tom","LastName":"Ripley"}]

Once again, let's look at how that same query is written in SQL. For this query, the SQL equivalent is a left outer join similar to the following:

```
SELECT a.Name, c.FirstName, c.LastName FROM Account a LEFT JOIN
Contact c ON (a.Id = c.AccountId)
```

The SQL query gets all Account records and any contacts associated with those accounts. The output returned in SQL isn't formatted as JSON. Other than that, the SQL and SOQL queries produce the same results.

At this point, you might be wondering whether SOQL supports aliasing. It does, but not how you're used to working with it in SQL. SOQL has no AS keyword. You can use aliases to represent object names in SOQL queries, but for field names, aliasing works only for aggregate queries, which we'll be covering next.

Remember, to dive deeper into this topic, check out the hands-on training video link in the Resources section.

As far as which functions you can use, SOQL has the ones you see in the table below. See the official docs (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_select_agg_functions.htm) for more specifics about each function.

SOQL Aggregate Functions

Function	Description
AVG()	Returns the average value of a numeric field.
COUNT() and COUNT(fieldName) and COUNT_DISTINCT()	Returns the number of rows matching the query criteria.
MIN()	Returns the minimum value of a field.
MAX()	Returns the maximum value of a field.
SUM()	Returns the total sum of a numeric field.

To get a record count for a particular table named Account in SQL, you do something like the following:

```
1 SELECT COUNT(*) FROM Account
```

In SOQL, this same query looks like this:

```
1 SELECT COUNT() FROM Account
```

Pretty similar, right?

Count(fieldName) is a newer version that returns the number of rows where the fieldName has a non-null value. What's different is that it returns that result as a list of AggregateResults and not as a single value.

Let's see a little of this in action.

Move from SQL to SOQL

1. In Developer Console, click the **Query Editor**.
2. Enter the following query:

   ```
   SELECT COUNT() FROM Account
   ```
   copy

3. Click **Execute**.
   The query results show the total number of rows with a number beside it.
4. Go back to the Query Editor tab and change the query to be this:

   ```
   SELECT COUNT(Id) FROM Account
   ```
   copy

5. Click **Execute**.
   Now the query results show only one row returned and a column that displays the total number of records.

Up until this point, we haven't talked much about how you handle the data you return from your SOQL queries. But why put off the inevitable? Let's roll up our sleeves and get down and dirty with some aggregate data.

We're going to jump into a little bit of Apex, but don't worry if it doesn't make complete sense yet. We'll be covering it more in detail later.

1. In Developer Console, select **Debug** and select **Open Execute Anonymous Window**.
2. Delete any existing code, and insert the following snippet:

```
List<AggregateResult> results  = [
    SELECT Industry, count(Id) total
    FROM Account GROUP BY Industry
];
for(AggregateResult ar : results) {
    System.debug('Industry: ' + ar.get('Industry'));
    System.debug('Total Accounts: ' + ar.get('total'));
}
```

Notice how we used an alias to represent the total along with the GROUP BY clause. In SOQL, you can only alias fields in aggregate queries that use the GROUP BY

Tell Me More

In addition to the GROUP BY clause (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_select_groupby.htm), SOQL offers other grouping clauses, such as GROUP BY ROLLUP, GROUP BY CUBE, and GROUPING. These clauses are useful for inspecting the results when a query returns values from several related tables. GROUP BY CUBE is a neat little clause that lets you add subtotals for all combinations of a grouped fields in the query results. To learn more about these clauses, look at the GROUP BY document in Resources.

Aggregate functions also support an optional HAVING clause (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_select_having.htm) that is similar to the HAVING Clause in SQL Server, so you should feel right at home with this one. It basically lets you filter the results that an aggregate function returns. Check out the Resources to learn more.

Resources

- *SOQL and SOSL Reference*: SOQL SELECT Syntax (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_select.htm)
- *SOQL and SOSL Reference*: Count and Count(fieldName) (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_select_count.htm)
- *Apex Developer Guide*: SOQL and SOSL Queries (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/langCon_apex_SOQL.htm)
- *SOQL and SOSL Reference*: GROUP BY (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_soql_select_groupby.htm)

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points

A You use SOQL only to perform queries using the SELECT statement.

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

B SOQL is object based, while SQL is record based.

Move from SQL to SOQL

C 'SELECT *' is not supported in SOQL.

D INSERT, UPDATE, and DELETE statements are not supported in SOQL.

E All of the above

---

2 What types of joins does SOQL support?

A SOQL supports only outer joins using the JOIN keyword.

B SOQL supports only inner joins using the JOIN keyword.

C SOQL supports both inner and outer joins using the JOIN keyword.

D SOQL does not support the JOIN keyword.

---

3 Which of the following is a Child-to-Parent query that returns a contact's first name, last name and account name?

B Contact

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

C SELECT FirstName, LastName, Account.Name FROM Contact JOIN Account

Move from SQL to SOQL

D SELECT FirstName, LastName, Account.Name FROM Contact

---

4 Which of the following is a Parent-to-Children query that returns an account's name and the first and last names of all associated contacts?

A SELECT Name, (Select FirstName, LastName FROM Contacts) FROM Account

B SELECT Name, (Select FirstName, LastName FROM Contact) FROM Account

C SELECT Name, FirstName, LastName FROM Account JOIN Contacts

D SELECT Name, Contact.FirstName, Contact.LastName FROM Account

---

5 Most SOQL aggregate functions typically return:

A A Boolean result type

B An AggregateResult type

SOQL records

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Second attempt earns 50 points. Three or more earns 25 points.

Move from SQL to SOQL

Check the Quiz to Earn 100 Points